Supervyse OPF Redfish CertificateService User Guide

Revision 1.0
Feb. 2025

Copyright (c) 2025, All Rights Reserved. Insyde Software.

No part of this publication may be reproduced, transmitted, transcribed, stored in a retrieval system, or translated into any language or computer language, in any form, or by any means, electronic, mechanical, magnetic, optical, chemical, manual or otherwise, without the prior written permission of Insyde Software Corp.

Disclaimer

Insyde Software provides this document and the programs "as is" without warranty of any kind, either expressed or implied, including, but not limited to, the implied warranties of merchantability or fitness for a particular purpose.

This document could contain technical inaccuracies or typographical errors. Changes are periodically made to the information herein; these changes will be incorporated in future revisions of this document. Insyde Software is under no obligation to notify any person of the changes.

The following trademarks are used in this document:

Insyde and InsydeH2O are registered trademarks or trademarks of Insyde Software. All other trademarks or trade names are property of their respective holders.

Content

Content 3

Revision History 4

1 Introduction 5
- 1.1 Overview 5
- 1.2 Related Information 5
- 1.3 Terms 5

2 Certificate Service Overview 6
- 2.1 What is Certificate? 6
- 2.2 Certificates in Supervyse OPF 7
- 2.3 Available Operations on Certificates 8

3 Read Certificates 9
- 3.1 Certificate Locations 9
- 3.2 GET Certificate 9

4 Create & Replace Certificates 11
- 4.1 Overview 11
- 4.2 Create Certificates 13
- 4.3 Replace Certificate 16

5 Generate CSR 18
- 5.1 Overview 18
- 5.2 Process 18

6 Delete Certificates 22
- 6.1 Certificate Locations 22
- 6.2 Delete Certificate 22

Revision History

Revision Number	Description	Author	Reviewer	Release Date
1.0	Initial release.	George Huang	Victor Wu	Feb, 2025

1 Introduction

1.1 Overview

This document is intended to guide developers, users, and testers to understand how to use the Certificate Service in Redfish for Supervyse OPF.

1.2 Related Information

The following publications and sources of information are referred to by this document or may be useful to you:

- *Supervyse OPF Redfish API User Guide, 1.4, Insyde Software, 2024*

1.3 Terms

The following terms are used throughout this document:

CA

Certificate Authority. A trusted entity that issues Secure Sockets Layer (SSL) certificates.

CSR

Certificate Signing Request. A message sent from an applicant to a CA in order to apply for a certificate.

PEM

Privacy-Enhanced Mail. The de facto file format for storing and sending cryptographic keys and certificates.

2 Certificate Service Overview

2.1 What is Certificate?

A Digital Certificate is a digital document that **binds a public key to an identity** (like a person, organization, or website). It is typically issued by a **Certificate Authority (CA)** and helps verify that the public key truly belongs to the specified entity, ensuring a secure connection.

X.509 is the international **standard** for public key certificates used in SSL/TLS (HTTPS), encryption, and authentication.

2.1.1 Example of an X.509 Certificate

Here's how an X.509 certificate looks in PEM format (Base64-encoded):

```
-----BEGIN CERTIFICATE-----
MIIDyzCCArOgAwIBAgIUWjLsvNCqez
... (Base64 encoded data) ...
-----END CERTIFICATE-----
```

You can view its content using OpenSSL:

```
$ openssl x509 -in certificate.pem -text –noout

Certificate:
   Data:
      ... (omitted) ...
      Issuer: C = US, O = Let's Encrypt, CN = R11
      Validity
         Not Before: Jul 27 19:26:19 2024 GMT
         Not After : Oct 25 19:26:18 2024 GMT
      Subject: CN = releases.ubuntu.com
      ... (omitted) ...
```

2.1.2 The three key fields

1. **Issuer**: The Certificate Authority (CA) that issued the certificate.
2. **Validity**: Defines the certificate's start and expiration dates.
3. **Subject**: The entity to which the certificate belongs (e.g., a domain like www.example.com).

2.2 Certificates in Supervyse OPF

There are five types of certificates in Supervyse OPF that can be managed via Redfish.

These certificates are categorized into **two** groups:

- Certificates used to **prove identity to others**:
    1. Web Server HTTPS Certificate:

       An HTTPS certificate for the web server (Redfish), used to identify itself to clients.

       URI: /Managers/<Manager_ID>/NetworkProtocol/HTTPS/Certificates/
    2. LDAP Certificate:

       Used to identify itself to an LDAP server.

       URI: /AccountService/LDAP/Certificates/
- Certificates used to **verify others**:
    1. VirtualMedia Certificate:

       Used to verify the remote server when inserting Virtual Media through HTTPS.

       URI: /Managers/<Manager_ID>/VirtualMedia/Oem/Insyde/Certificates/
    2. System Boot HTTPS Certificate:

       Used to verify the remote booting of this computer system via HTTPS.

       URI: /Systems/<System_ID>/Boot/Certificates/
    3. Truststore Certificate:

       Used to store certificates trusted by the BMC, aside from the types listed above. It serves as a place for other services to store certificates required for their functionality (e.g., the LDAP service stores certificates here to verify the LDAP server).

       URI: /Managers/<Manager_ID>/Truststore/Certificates/

2.3 Available Operations on Certificates

Users can perform **CRUD** operations on the certificates:

- **Create** (POST): Install a new certificate.
- **Read** (GET): Retrieve a certificate's content.
- **Update** (Replace Action): Replace an existing certificate.
- **Delete** (DELETE): Remove a certificate.

3 Read Certificates

3.1 Certificate Locations

Users can read a list of installed certificates by sending a **GET** request to:
/redfish/v1/CertificateService/CertificateLocations/

3.1.1 Example

Here's an example using cURL.

3.1.1.1 Request

```
$ curl -ku root:0penBmc -H 'Content-Type: application/json' -X GET 'https://<BMC_IP>/redfish/v1/
CertificateService/CertificateLocations/'
```

3.1.1.2 Response


```
{
  "@odata.id": "/redfish/v1/CertificateService/CertificateLocations",
  "@odata.type": "#CertificateLocations.v1_0_4.CertificateLocations",
  "Description": "Defines a resource that an administrator can use in order to locate all certificates
installed on a given service",
  "Id": "CertificateLocations",
  "Links": {
    "Certificates": [
      {
        "@odata.id": "/redfish/v1/Managers/BMC_0/NetworkProtocol/HTTPS/Certificates/1"
      }
    ],
    "Certificates@odata.count": 1
  },
  "Name": "Certificate Locations"
}
```


3.2 GET Certificate

Users can read a certificate's content by sending a **GET** request to its URL.

3.2.1 Example

Here's an example using cURL.

3.2.1.1 Request

```
$ curl -ku root:0penBmc -H 'Content-Type: application/json' -X GET 'https://<BMC_IP>/redfish/ v1/
Managers/BMC_0/NetworkProtocol/HTTPS/Certificates/1'
```

3.2.1.2 Response


```
{
  "@odata.id": "/redfish/v1/Managers/BMC_0/NetworkProtocol/HTTPS/Certificates/1",
  "@odata.type": "#Certificate.v1_9_0.Certificate",
  "CertificateString": "-----BEGIN CERTIFICATE-----\n... (Base64 encoded data) ...\n-----END
CERTIFICATE-----\n",
  "Description": "HTTPS Certificate",
  "Id": "1",
  "Issuer": {
    "CommonName": "Insyde",
    "Country": "US",
    "DomainComponents": [],
    "Organization": "Insyde Software Corp.",
    "OrganizationalUnit": "BMC Software Team",
    "State": "MA"
  },
  "KeyUsage": [],
  "Name": "HTTPS Certificate",
  "Subject": {
    "CommonName": "Insyde",
    "Country": "US",
    "DomainComponents": [],
    "Organization": "Insyde Software Corp.",
    "OrganizationalUnit": "BMC Software Team",
    "State": "MA"
  },
  "ValidNotAfter": "2026-01-01T00:00:00+00:00",
  "ValidNotBefore": "1970-01-01T00:00:00+00:00"
}
```


3.2.1.3 Essential Properties

Property	Description
CertificateString	The Base64-encoded string for the certificate.
Issuer	The issuer of the certificate.
Subject	The subject of the certificate.

Note: For detailed descriptions of additional properties, please refer to *Supervyse OPF Redfish API User Guide*.

4 Create & Replace Certificates

4.1 Overview

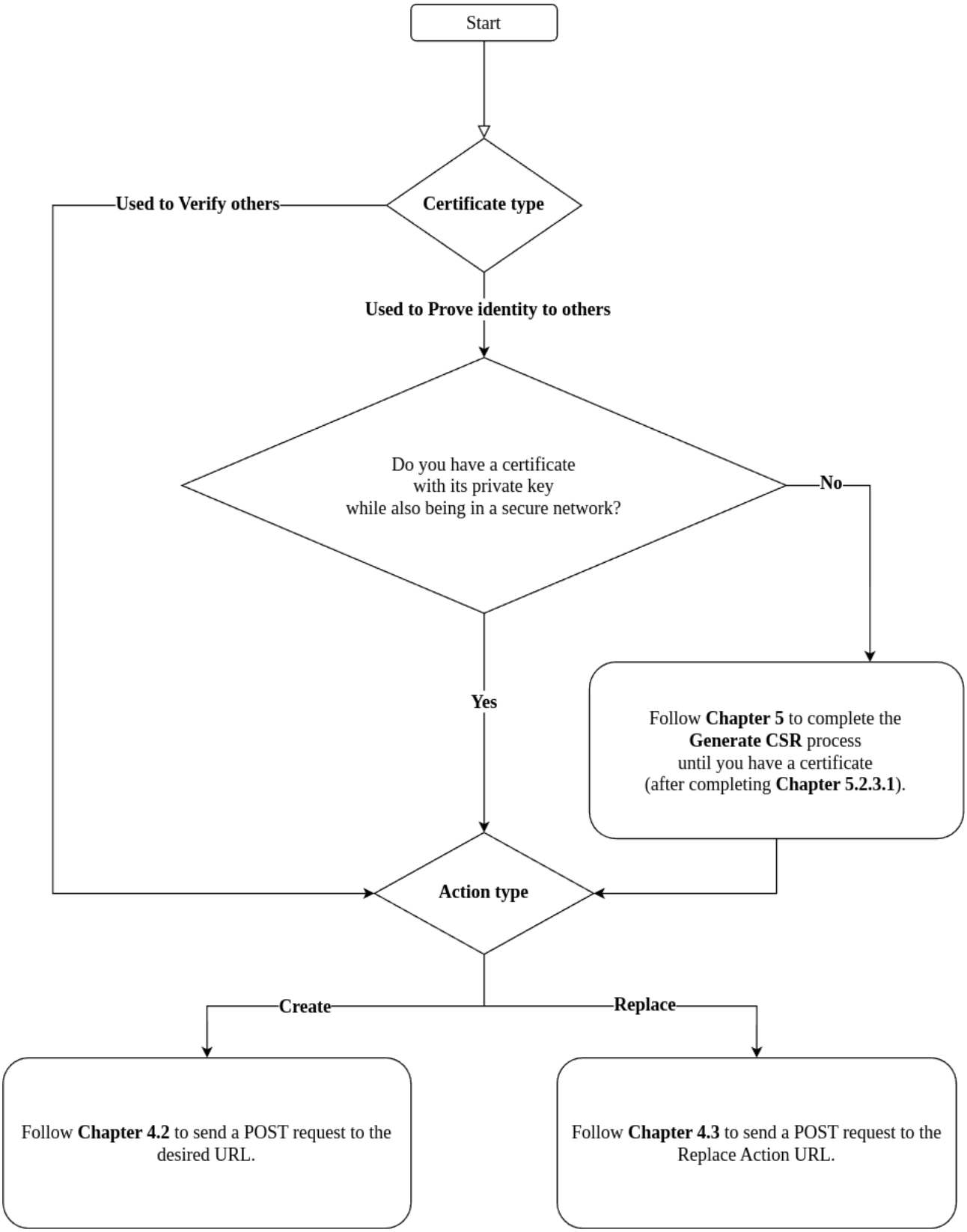
For Certificates used to **prove identity to others** (Chapter 2.2), users must install the certificate along with its private key. Otherwise, the installation will fail.

To achieve this, users can either:

- Use the **Generate CSR** Action to generate a private key on the web server first, refer to **Generate CSR** (Chapter 5) for more details.
- Install a certificate that includes the private key directly (**not recommended** due to potential security risks—only do this in a secure network). Below is a comparison between a **PEM file with a private key** and one **without a private key**.

4.1.1 Certificate Installation Process

On the next page, here's a flowchart that better demonstrates the certificate installation process.


Start
Certificate type
Used to Verify others
Used to Prove identity to others
Do you have a certificate
with its private key
while also being in a secure network?
No
Yes
Follow Chapter 5 to complete the
Generate CSR process
until you have a certificate
(after completing Chapter 5.2.3.1).
Action type
Create
Replace
Follow Chapter 4.2 to send a POST request to the
desired URL.
Follow Chapter 4.3 to send a POST request to the
Replace Action URL.

4.1.2 Examples of PEM

Here's a comparison of a certificate with and without its private key in PEM format to help users determine the type of certificate they have.

4.1.2.1 Without a private key

```
-----BEGIN CERTIFICATE-----
MIIFDzCCA/egAwIBAgISBN2WK83AFzOq09TFfxSxtlZuMA0GCSqGSIb3DQEBCwUA
... (Base64 encoded data) ...
-----END CERTIFICATE-----
```

4.1.2.2 With a private key:

```
-----BEGIN CERTIFICATE-----
MIIErDCCA5SgAwIBAgIUATtIgY2a4S4LEyLZNnBJNTU6ANswDQYJKoZIhvcNAQEL
... (Base64 encoded data) ...
-----END CERTIFICATE-----
-----BEGIN RSA PRIVATE KEY-----
MIIEowIBAAKCAQEA08SlxADJDwurYjvVAlFXq8jniIWgTJ9LaG8XkTL4wLBb2Kpr
... (Base64 encoded data) ...
-----END RSA PRIVATE KEY-----
```

4.2 Create Certificates

4.2.1 Overview

Users can install new certificates for the following types by sending a **POST** request to the respective URL:

- LDAP Certificates:

  /redfish/v1/AccountService/LDAP/Certificates/

- Truststore Certificate:

  /redfish/v1/Managers/<Manager_ID>/Truststore/Certificates/

- VirtualMedia Certificate:

  /redfish/v1/Managers/<Manager_ID>/VirtualMedia/<Device_ID>

  /Actions/Oem/InsydeOEMExtensions.PostCertificate/

- System Boot HTTPS Certificate:

  /redfish/v1/Systems/<System_ID>/Boot/Certificates/

4.2.1.1 How to install Web Server HTTPS Certificate

Normally, installing a Web Server HTTPS Certificate is **prohibited** because the web server shall automatically generate a default certificate when the system starts, and only **one** Web Server HTTPS Certificate can exist at any given time.

However, users can still install a Web Server HTTPS Certificate by using **Replace** instead of Create.

4.2.2 Send POST request

To send a POST request, users can either:

1. Attach a raw PEM file to the request.
2. Include the certificate as a property in the JSON request.

The following sections demonstrate both methods:

4.2.2.1 Example

Here's some examples using cURL.

4.2.2.1.1 Attach a raw PEM file

IMPORTANT: When sending a **POST** request with a PEM file using **cURL**, you must use the **--data-binary** flag to prevent data corruption.

```
$ curl -ku root:0penBmc -X POST
'https://<BMC_IP>/redfish/v1/Managers/BMC_0/VirtualMedia/0/Actions/Oem/InsydeOEMExtensions.PostCertificate' --data-binary @myCert.pem
```

4.2.2.1.2 Sending with JSON

Action Parameters

Property	Type	Description	Attributes
CertificateString	string	The certificate content as a string.	required
CertificateType	string	The format of the certificate. The only allowable value is "PEM" for now.	required

```
$ cat cert_install.json
{
  "CertificateString": "-----BEGIN CERTIFICATE-----\n... (Base64 encoded data) ...\n-----END CERTIFICATE-----",
  "CertificateType": "PEM"
}

$ curl -ku root:0penBmc -H 'Content-Type: application/json' -X POST
'https://<BMC_IP>/redfish/v1/Managers/BMC_0/VirtualMedia/0/Actions/Oem/InsydeOEMExtensions.PostCertificate' -d @cert_install.json
```

4.2.2.1.3 Successful response


```
{
  "@odata.id": "/redfish/v1/Managers/BMC_0/VirtualMedia/Oem/Insyde/Certificates/0",
  "@odata.type": "#Certificate.v1_9_0.Certificate",
  "CertificateString": "-----BEGIN CERTIFICATE-----\n... (Base64 encoded data) ...\n-----END
CERTIFICATE-----\n",
  "Description": "VM Certificate",
  "Id": "0",
  "Issuer": {
    "CommonName": "R11",
    "Country": "US",
    "DomainComponents": [],
    "Organization": "Let's Encrypt"
  },
  "KeyUsage": [
    "KeyEncipherment",
    "ServerAuthentication",
    "ClientAuthentication"
  ],
  "Name": "VM Certificate",
  "Subject": {
    "CommonName": "releases.ubuntu.com",
    "DomainComponents": []
  },
  "ValidNotAfter": "2024-10-25T19:26:18+00:00",
  "ValidNotBefore": "2024-07-27T19:26:19+00:00"
}
```

4.3 Replace Certificate

4.3.1 Overview

Users can replace existing certificates by sending a **POST** request to:
/redfish/v1/CertificateService/Actions/CertificateService.ReplaceCertificate/

4.3.2 Action Parameters

Property	Type	Description	Attributes
CertificateString	string	The certificate content as a string.	required
CertificateType	string	The format of the certificate. The only allowable value is "PEM" for now.	required
CertificateUri	string	The URL of the certificate being replaced.	required

4.3.3 Example

Here's an example using cURL.

4.3.3.1 Request

```
$ cat cert_replace.json
 {
  "CertificateString": "-----BEGIN CERTIFICATE-----\n... (Base64 encoded data) ...\n-----END
CERTIFICATE-----",
  "CertificateType": "PEM",
  "CertificateUri": {
    "@odata.id": "/redfish/v1/Managers/BMC_0/VirtualMedia/Oem/Insyde/Certificates/0"
  }
}

$ curl -ku root:0penBmc -H 'Content-Type: application/json' -X POST 'https://<BMC_IP>/redfish/
v1/CertificateService/Actions/CertificateService.ReplaceCertificate/' -d @cert_replace.json
```

4.3.3.2 Response

```
{
  "@odata.id": "/redfish/v1/Managers/BMC_0/VirtualMedia/Oem/Insyde/Certificates/0",
  "@odata.type": "#Certificate.v1_9_0.Certificate",
  "CertificateString": "-----BEGIN CERTIFICATE-----\n... (Base64 encoded data) ...\n-----END
CERTIFICATE-----\n",
  "Description": "VM Certificate",
  "Id": "0",
  "Issuer": {
    "CommonName": "R11",
    "Country": "US",
    "DomainComponents": [],
    "Organization": "Let's Encrypt"
  },
```

```
  "KeyUsage": [
    "KeyEncipherment",
    "ServerAuthentication",
    "ClientAuthentication"
  ],
  "Name": "VM Certificate",
  "Subject": {
    "CommonName": "releases.ubuntu.com",
    "DomainComponents": []
  },
  "ValidNotAfter": "2024-10-25T19:26:18+00:00",
  "ValidNotBefore": "2024-07-27T19:26:19+00:00"
}
```

5 Generate CSR

5.1 Overview

In an **insecure** network, it's recommended to install certificates using **Generate CSR** (Certificate Signing Request).

This section will guide you through the step-by-step process.

5.2 Process

To create and install a self-signed certificate on BMC via Redfish, follow these steps:

1. Create your own SSL Certificate Authority (CA).
2. Generate the CSR.
3. Create a CA-signed certificate using the CSR.
4. Install the CA-signed certificate.

5.2.1 Create your own CA

5.2.1.1 Create a private key for your CA

```
$ openssl genrsa -out myCA.key
```

5.2.1.2 Create a root certificate for your CA

```
$ openssl req -x509 -new -key myCA.key -out myCA.pem
```

This will start an interactive script (below) to enter information about your CA.

For this mock CA, you can enter any information here.

```
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:
State or Province Name (full name) [Some-State]:
Locality Name (eg, city) []:
Organization Name (eg, company) [Default Company Ltd]:
Organizational Unit Name (eg, section) []:
Common Name (eg, YOUR name) []:
Email Address []:
```

5.2.2 Generate CSR (Certificate Signing Request)

5.2.2.1 Create CSR file

Create a CSR file (csr_file.json) with all the following fields.

```
{
   "City": <City Name>,
   "CertificateCollection": {
      "@odata.id": "/redfish/v1/Managers/BMC_0/NetworkProtocol/HTTPS/Certificates/"
   },
   "CommonName": <Common Name>,
   "Country": <Country Name>,
   "Organization": <Organization Name>,
   "OrganizationalUnit": <Organization Unit Name>,
   "State": <State Name>
}
```

Note: The information here serves a different purpose than the one in the last step. The previous step contained information of your mock CA, whereas this step pertains to the BMC webserver.

Note: The CertificateCollection field must match your desired certificate type.

5.2.2.1.1 Example JSON

```
{
   "City": "Austin",
   "CertificateCollection": {
      "@odata.id": "/redfish/v1/Managers/BMC_0/NetworkProtocol/HTTPS/Certificates/"
   },
   "CommonName": "xx.xx.xx.xx",
   "Country": "US",
   "Organization": "ABC Limited",
   "OrganizationalUnit": "IT",
   "State": "AU"
}
```

5.2.2.2 Request Generate CSR Action

Generate the CSR from BMC by sending a **POST** request to:
`/redfish/v1/CertificateService/Actions/CertificateService.GenerateCSR/`

5.2.2.2.1 Example request

```
$ curl -ku root:0penBmc -H 'Content-Type: application/json' -X POST 'https://<BMC_IP>/redfish/ v1/
CertificateService/Actions/CertificateService.GenerateCSR/' -d @csr_file.json
```

5.2.2.2.2 Example response

```
{
  "CSRString": "-----BEGIN CERTIFICATE REQUEST-----\n... (Base64 encoded data) ...\n-----END
CERTIFICATE REQUEST-----\n",
  "CertificateCollection": {
```

```
  "@odata.id": "/redfish/v1/Managers/BMC_0/NetworkProtocol/HTTPS/Certificates/"
}
```

5.2.2.3 Convert response

Convert the response into a .csr file (bmc.csr).

Note: You must **manually** replace \n with actual newline characters.

```
$ cat bmc.csr
-----BEGIN CERTIFICATE REQUEST-----
MIICyzCCAbMCAQEwgYUxDzANBgNVBAcMBkF1c3RpbjEUMBIGA1UEAwwLeHgueHgu
... (Base64 encoded data) ...
-----END CERTIFICATE REQUEST-----
```

5.2.3 Create CA-signed Certificate

5.2.3.1 Sign CSR

Use the CA you created earlier to signed the CSR (bmc.csr), thereby creating a CA-signed certificate (my.pem).

```
$ openssl x509 -req -in bmc.csr -CA myCA.pem -CAkey myCA.key -CAcreateserial -out my.pem
```

5.2.3.2 Convert to JSON request

Create a JSON file (certificate.json) with the CA-signed certificate created in step 1.

Note: You must **manually** replace newline characters with \n.

The JSON file should **follow the payload format** mentioned earlier in the Create/Replace Certificates section.

```
{
  "CertificateString": "-----BEGIN CERTIFICATE-----\n... (Base64 encoded data) ...\n-----END
CERTIFICATE-----",
  "CertificateType": "PEM",
  "CertificateUri":
  {
    "@odata.id": "/redfish/v1/Managers/BMC_0/NetworkProtocol/HTTPS/Certificates/1"
  }
}
```

5.2.4 Install Certificate

5.2.4.1 Replace or Install Certificate

Replace or install your certificate using the earlier JSON file (certificate.json).

5.2.4.1.1 Example request

```
$ curl -ku root:0penBmc -H 'Content-Type: application/json' -X POST 'https://<BMC_IP>/redfish/ v1/
CertificateService/Actions/CertificateService.ReplaceCertificate/' -d @certificate.json
```

5.2.4.1.2 Example response


```
{
  "@odata.context": "/redfish/v1/$metadata#Certificate.Certificate",
  "@odata.id": "/redfish/v1/Managers/<Manager_ID>/NetworkProtocol/HTTPS/Certificates/1",
  "@odata.type": "#Certificate.v1_0_0.Certificate",
  "CertificateString": "-----BEGIN CERTIFICATE-----\n... (Base64 encoded data) ...\n-----END
CERTIFICATE-----\n",
  "Description": "HTTPS certificate",
  "Id": "1",
  "Issuer": {
    "City": "Portland",
    "CommonName": "XYZ CERTIFICATE AUTHORITY",
    "Country": "US",
    "Organization": "XYZ",
    "OrganizationalUnit": "IT",
    "State": "Oregon"
  },
  "KeyUsage": [],
  "Name": "HTTPS certificate",
  "Subject": {
    "City": "Austin",
    "CommonName": "xx.xx.xx.xx",
    "Country": "US",
    "Organization": "ABC Limited",
    "State": "AU"
  },
  "ValidNotAfter": "2021-01-23T21:13:40+00:00",
  "ValidNotBefore": "2019-09-11T21:13:40+00:00"
}
```

6 Delete Certificates

6.1 Certificate Locations

Users can read a list of installed certificates by sending **GET** request to:
/redfish/v1/CertificateService/CertificateLocations/

6.1.1 Example

Here's an example using cURL.

6.1.1.1 Request

```
$ curl -ku root:0penBmc -H 'Content-Type: application/json' -X GET 'https://<BMC_IP>/redfish/ v1/
CertificateService/CertificateLocations/'
```

6.1.1.2 Response


```
{
  "@odata.id": "/redfish/v1/CertificateService/CertificateLocations",
  "@odata.type": "#CertificateLocations.v1_0_4.CertificateLocations",
  "Description": "Defines a resource that an administrator can use in order to locate all certificates
installed on a given service",
  "Id": "CertificateLocations",
  "Links": {
    "Certificates": [
      {
        "@odata.id": "/redfish/v1/Managers/BMC_0/NetworkProtocol/HTTPS/Certificates/1"
      }
    ],
    "Certificates@odata.count": 1
  },
  "Name": "Certificate Locations"
}
```


6.2 Delete Certificate

Users can delete a certificate by sending a **DELETE** request to its URL.

6.2.1 Example

Here's an example using cURL.

6.2.1.1 Request

```
$ curl -ku root:0penBmc -H 'Content-Type: application/json' -X DELETE 'https://<BMC_IP>/redfish/
v1/AccountService/LDAP/Certificates/1'
```

6.2.1.2 Response


```
{
 "@Message.ExtendedInfo": [
  {
   "@odata.type": "#Message.v1_1_2.Message",
   "Message": "The request completed successfully.",
   "MessageArgs": [],
   "MessageId": "Base.1.16.Success",
   "MessageSeverity": "OK",
   "Resolution": "None"
  }
 ]
}
```